TP 3 Projet Fédéré
Automatisation Jira + GitHub

Objectif : À la fin du TP, vous serez capables de :

- Relier un ticket Jira à du code automatiquement
- Déclencher des changements de statut sans action manuelle
- Garantir la traçabilité User Story → Code → Pull Request → Done
- Sécuriser la branche principale
- Gestion automatisée des versions via GitHub Actions

Partie 1 : Vérifier l'intégration

Dans Jira :

1. Aller dans **Project Settings**
2. Ouvrir **Development tools**
3. Vérifier que GitHub est connecté

Validation :

Créer un ticket test : `WEB-XX`

Puis :

1. Créer une branche dans GitHub : `FrontEnd/WEB-XX-test-automation`
2. Rafraîchir Jira.

Le ticket doit afficher la branche associée.

Partie 2 : Convention obligatoire ~~ *Comprendre que l'automatisation dépend du nommage.*

1. Sur la branche `FrontEnd/WEB-XX-test-automation`
   - Faire un commit avec le message "`WEB-XX : ajout bouton test`"
   - Push sur GitHub.
   - Observer le ticket Jira.
2. Tester sans ID
   - Faire un commit avec le message : "`ajout correction css`"
   - Push.

Réflexion

1. Que se passe-t-il dans Jira ?
2. Pourquoi le deuxième commit n'est-il pas lié ?
3. Quelle règle faut-il imposer à l'équipe ?

Partie 3 : Automatisation des statuts des US ~~ *Ne plus changer les statuts manuellement*

Règle automatique — Passage en "In Progress"

1. Dans Jira → Automation → Create Rule
   - Trigger : Branch created
   - Action : Transition issue → In Progress
2. Test :
   - Créer un nouveau ticket WEB-XX
   - Sur Github : Créer branche FrontEnd/WEB-XX-login-page
   - Vérifier si le ticket change automatiquement de statut

Règle — Passage en Done au merge

3. Dans Jira → Automation → Create Rule
   - Trigger : Pull request merged
   - Action : Transition issue → Done
   - Ajouter commentaire : "Développement terminé et mergé dans main."
4. Test :
   - Créer la page login.html dans le dossier local
   - Via Git Bash :
     - Transiter vers la branche déjà créée : `git checkout -b FrontEnd/WEB-XX-login-page`
     - Ajoute le fichier : `git add login.html`
     - Commiter : `git commit -m "WEB-XX : Création page login"`
     - Push sur Github : `git push -u origin FrontEnd/WEB-XX-login-page`
   - Créer le Pull Request et vérifier les changements relatifs au ticket sur Jira
   - Merger le Pull Request et vérifier les changements relatifs au ticket sur Jira

Partie 4 : Protection de la branche main ~~ *Empêcher les erreurs humaines.*

Dans GitHub :

1. Settings
2. Branches
3. Dans **Branch protection rules**, clique sur **Add classic branch protection rule**.

   Configurer :
   - **Branch name pattern** : main
   - Cocher les options suivantes :
     - **Require a pull request before merging**
     - **Require approvals** (ou "Require at least 1 review")
     - Sauvegarder la règle
4. Vérifier la protection :
   - Dans ton dépôt local, teste le push direct :

```
git checkout main
git commit --allow-empty -m "test push direct"
git push
```

Partie 5 : Tag incrémental automatique à chaque merge sur main ~~ *Créer des versions à chaque MERGE d'un PULL REQUEST sur la branche main.*

Créer le fichier de workflow

Dans le dépôt GitHub :

1. Créer le dossier (s'il n'existe pas) : .github/workflows/
2. Créer le fichier : tag-version.yml
3. Copier le contenu suivant :

```
name: Auto Tag Version
on:
  push:
    branches:
      - main

permissions:
  contents: write

jobs:
  tag:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v3
        with:
          fetch-depth: 0

      - name: Get latest tag
        run: |
          TAG=$(git describe --tags --abbrev=0 2>/dev/null || echo "v0.0")
          echo "LATEST_TAG=$TAG" >> $GITHUB_ENV
          echo "Current tag: $TAG"

      - name: Bump patch version
        run: |
          IFS='.' read -r MAJ MIN PATCH <<< "${LATEST_TAG#v}"
          PATCH=$((PATCH+1))
          NEW_TAG="v$MAJ.$MIN.$PATCH"
          echo "NEW_TAG=$NEW_TAG" >> $GITHUB_ENV
          echo "New tag: $NEW_TAG"

      - name: Create and push tag
        run: |
          git tag $NEW_TAG
          git push origin $NEW_TAG
```

4. Déclencher l'automatisation
   - Créer une branche : git checkout -b test-tag
   - Faire un commit : git commit --allow-empty -m "Test auto tag"
   - Push la branche
   - Ouvrir une Pull Request
   - Merge dans main
5. Vérifier le tag
   - Dans GitHub :
     - Aller dans **Code → Tags**
     - Vérifier qu'un nouveau tag est créé automatiquement. Exemple : `v0.0.1` puis `v0.0.2` puis `v0.3`